

Name

Class

Date

OOP - 12

Total questions: 11

Worksheet time: 6mins

1.

```
class A {
public:
    virtual void f() {cout << "A";}
};

class B : public A {
public:
    void f() override {cout << "B";}
};

class C : public B {
};

int main() {
    C obj;
    obj.f();
}
```

a) A

b) B

c) Няма да се компилира!

2.

```
class A {
public:
    virtual void f() {cout << "A";}
};

class B : public A {
public:
    void f() final {cout << "B";}
};

class C : public B {
    void f() override {cout << "C";}
};

int main() {
    C obj;
    obj.f();
}
```

a) A

b) B

c) C

d) Няма да се компилира!

3.

```
class A {
    int a = 3;
public:
    virtual void f() = 0;
    int getA() const {return a;}
};

int main() {
    A obj;
    obj.getA();
}
```

a) 3

b) 0

c) Няма да се компилира!

4.

```
class A {
    int a = 3;
public:
    virtual void f() = 0;
    int getA() const {return a;}
};

class B : public A {
    int b = 5;
public:
    int getB() const {return b;}
};

int main() {
    B obj;
    obj.getB();
}
```

a) 3

b) 5

c) Няма да се компилира!

5.

```
class Base {
public: virtual void f() {cout << "0 ";}; };

class Derived1 : public Base {
public: void f() {cout << "1 ";}; };

class Derived2 : public Base {
public: void f() {cout << "2 ";}; };

int main() {
    Derived1 der1;
    Derived2 der2;
    Base** arr = new Base*[2];
    arr[0] = &der1;
    arr[1] = &der2;
    for (int i = 0; i < 2; i++) {
        arr[i]->f();
    }
}
```

a) 0 0

b) 1 2

c) 1 1

d) Няма да се компилира!

6.

```
class A {  
public: int a = 0;};  
  
class B : public A {  
public: int b = 1;};  
  
class C : public A {  
public: int c = 1;};  
  
class D : public B, public C {  
public: int d = 2;};  
  
int main() {  
    D obj;  
    cout << obj.a;  
}
```

a) 0

b) 1

c) 2

d) Няма да се компилира!

7.

```
class Base {  
public: void f() {cout << "Base";};};  
  
class Derived: public Base {  
public: void f() {cout << "Derived";};};  
  
int main() {  
    Base* bPtr = new Derived;  
    bPtr->f();  
}
```

a) Base

b) Derived

c) Няма да се компилира!

8.

```
class Base {
public:
    virtual Base() {cout << "Base()";}
    virtual void f() {cout << "B f";}
};

class Derived : public Base {
public:
    Derived() : Base() {cout << "Derived()";}
    void f() override {cout << "D f";}
};

int main() {
    Derived obj;
    obj.f();
}
```

- a) Base()
Derived()
D f
- c) Base()
Derived()
B f

- b) Derived()
Base()
D f
- d) Няма да се компилира!

9.

```
class Base {
public:
    Base() {cout << "Base() ";}
    ~Base() {cout << "~Base() ";}
};

class Derived : public Base {
public:
    Derived() {cout << "Derived() ";}
    ~Derived() {cout << "~Derived() ";}
};

int main() {
    Base* bPtr = new Derived;
    delete bPtr;
}
```

- a) Base()
Derived()
~Base()
- c) Base()
Derived()

- b) Base()
Derived()
~Derived()
~Base()
- d) Няма да се компилира!

10.

```
class Base {
public:
    virtual void f() { cout << "Base"; }
};

class Derived : public Base {
public:
    void f() override { cout << "Derived"; }
};

void g(Base obj) {
    obj.f();
}

int main() {
    Derived d;
    g(d);
}
```

a) Base

b) Derived

c) Няма да се компилира!

11.

```
class Base {
public:
    virtual ~Base() = 0;
};

Base::~~Base() {
    cout << "~Base() ";
}

class Derived : public Base {
public:
    void f() { cout << "f "; }
};

int main() {
    Derived d;
    d.f();
}
```

a) f

b) f ~Base()

c) Няма да се компилира!